

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ  
федеральное государственное автономное образовательное учреждение высшего  
образования  
"Омский государственный технический университет"  
Кафедра «Информатика и вычислительная техника»

**Отчет**  
**по лабораторной работе № 5**  
дисциплина «Проектирование и тестирование программного обеспечения»

|           |                             |
|-----------|-----------------------------|
| Выполнил: | Шмидт А.В.<br>гр. ИВТ - 244 |
| Проверил: | асс. Симоненко А.Е.         |

Омск 2026

## Задача 9.4 (Кубики)

Задание:

У вас есть  $N$  цветных кубиков разного веса. Каждая грань кубика покрашена в свой цвет. Вам нужно построить максимально высокую башню, подчиняющуюся тем ограничениям, что (1) мы не можем класть тяжелый кубик на легкий и (2) цвет нижней грани каждого кубика (за исключением самого нижнего) должен совпадать с цветом верхней грани кубика под ним.

Входные данные могут содержать несколько тестовых блоков. Первая строка каждого тестового блока содержит целое число  $N$  ( $1 \leq N \leq 500$ ), задающее число данных вам кубиков,  $i$ -я строка из следующих  $N$  содержит описание  $i$ -го кубика. Кубик описывается цветами его граней в следующем порядке: передняя, задняя, левая, правая, верхняя и нижняя. Для удобства оттенки цветов описываются целыми числами от 1 до 100. Кубики вводятся в порядке их утяжеления, то есть кубик 1 самый легкий, а кубик  $N$  самый тяжелый.

Входные данные завершаются при равном 0.

Для каждого блока начните с вывода строки, содержащей номер тестового блока, как показано в примере выходных данных. На следующей строке выведите число кубиков в самой высокой возможной башне. Далее выведите описания кубиков в башне, по одному описанию на строку. Каждое описание содержит порядковый номер этого кубика во входных данных, один пробел, а затем строку-идентификатор (front, back, left, right, top, bottom) верхней грани кубика в башне. Может существовать несколько решений, подойдет любое.

Выходные данные для двух последовательных блоков должны быть разделены пустой строкой.

## Схема алгоритма

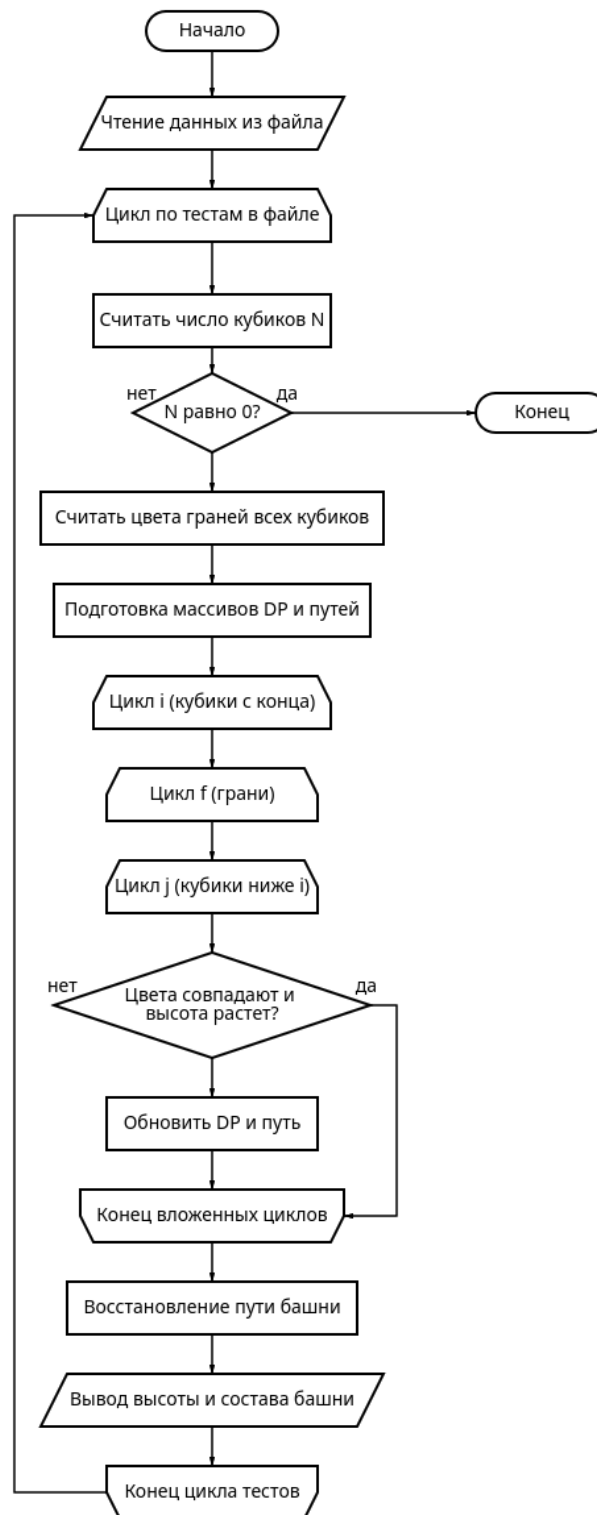


Рисунок 1 - Схема алгоритма.

Тестовые наборы:

Тестовый набор 1

Входные данные:

10

1 5 10 3 6 5

2 6 7 3 6 9

5 7 3 2 1 9

1 3 3 5 8 10

6 6 2 2 4 4

1 2 3 4 5 6

10 9 8 7 6 5

6 1 2 3 4 7

1 2 3 3 2 1

3 2 1 1 2 3

0

Выходные данные:

Case #1

8

1 bottom

2 back

3 front

4 top

5 front

6 front

8 front

9 front

Тестовый набор 2

Входные данные:

3

1 2 2 2 1 2

3 3 3 3 3 3

3 2 1 1 1 1

0

Выходные данные:

Case #2

2

2 front

3 front

Тестовый набор 3

Входные данные:

1

1 2 3 4 5 6

0

Выходные данные:

Case #3

1

1 front

Тестовый набор 4

Входные данные:

2

1 1 1 1 1 1

2 2 2 2 2 2

0

Выходные данные:

Case #4

1

1 front

Тестовый набор 5

Входные данные:

2

5 5 5 5 1 2

2 2 2 2 2 2

0

Выходные данные:

Case #5

2

1 top

2 front

Тестовый набор 6

Входные данные:

3

10 10 10 10 1 10

20 20 20 20 10 20

20 20 20 20 20 20

0

Выходные данные:

Case #6

3

1 top

2 top

3 front

Результат прохождения тестов программой:

```
===== test session starts =====
collecting ... collected 6 items

task9_4-tests.py::test_solve_tower_height[10-cubes0-8] PASSED [ 16%]
task9_4-tests.py::test_solve_tower_height[3-cubes1-2] PASSED [ 33%]
task9_4-tests.py::test_solve_tower_height[1-cubes2-1] PASSED [ 50%]
task9_4-tests.py::test_solve_tower_height[2-cubes3-1] PASSED [ 66%]
task9_4-tests.py::test_solve_tower_height[2-cubes4-2] PASSED [ 83%]
task9_4-tests.py::test_solve_tower_height[3-cubes5-3] PASSED [100%]

===== 6 passed in 0.21s =====
```

Рисунок 2 - Скриншот выполнения тестов.

### Задача 10.4 (Постановщик задач)

Задание:

В этом году так много студентов решили принять участие в региональных этапах олимпиад по программированию, что в жюри решили составить отбраковочные тесты для того, чтобы выявить наиболее многообещающих студентов. В этот тест могут входить до 100 задач из 20 категорий. Вам предложили набрать задачи для этого теста.

Поначалу работа казалась легкой, так как вам сказали, что дадут набор примерно из 1000 задач, поделенный на соответствующие категории. После того как вы получили задачи, оказалось, что их авторы нередко относили одну задачу к нескольким категориям. Так как ни одну из задач нельзя использовать в тесте более одного раза и число задач каждой категории установлено заранее, то выбрать задачи для теста - не такая уж и легкая работа.

Входной файл может содержать несколько тестовых блоков, каждый из которых начинается со строки, содержащей два целых числа,  $pk$  и  $pr$ , где  $pk$  - это количество категорий, а  $pr$  - число задач в данном наборе. Число категорий лежит в диапазоне от 2 до 20, а число задач в наборе не превышает 1000.

Вторая строка содержит  $pk$  положительных целых чисел, причем  $i$ -е число задает количество задач, которые должны быть отнесены к категории  $i$  ( $1 \leq i \leq pk$ ). Вы можете считать, что сумма этих чисел не превышает 100.

$j$ -я из следующих  $pr$  строк содержит информацию о категориях  $j$ -й задачи набора. Каждое такое описание категорий задачи начинается с положительного целого числа, задающего количество категорий, к которым может быть отнесена задача. За этим числом следуют сами номера категорий.

Входные данные завершаются тестовым блоком, который содержит нули в качестве  $pk$  и  $pr$ .

Для каждого тестового блока выведите строку, сообщающую, можно ли набрать задачи из набора, следуя заданным ограничениям. Для положительного ответа используйте 1, для отрицательного - 0.

Если задачи набрать можно, выведите еще  $pk$  строк так, чтобы  $i$ -я строка содержала номера задач, которые могут быть включены в категорию  $i$ . Номера задач должны быть положительными целыми числами, не превышающими  $pr$ , и два последовательных номера должны быть разделены одним пробелом. Подойдет любая правильная выборка задач.

Схема алгоритма:

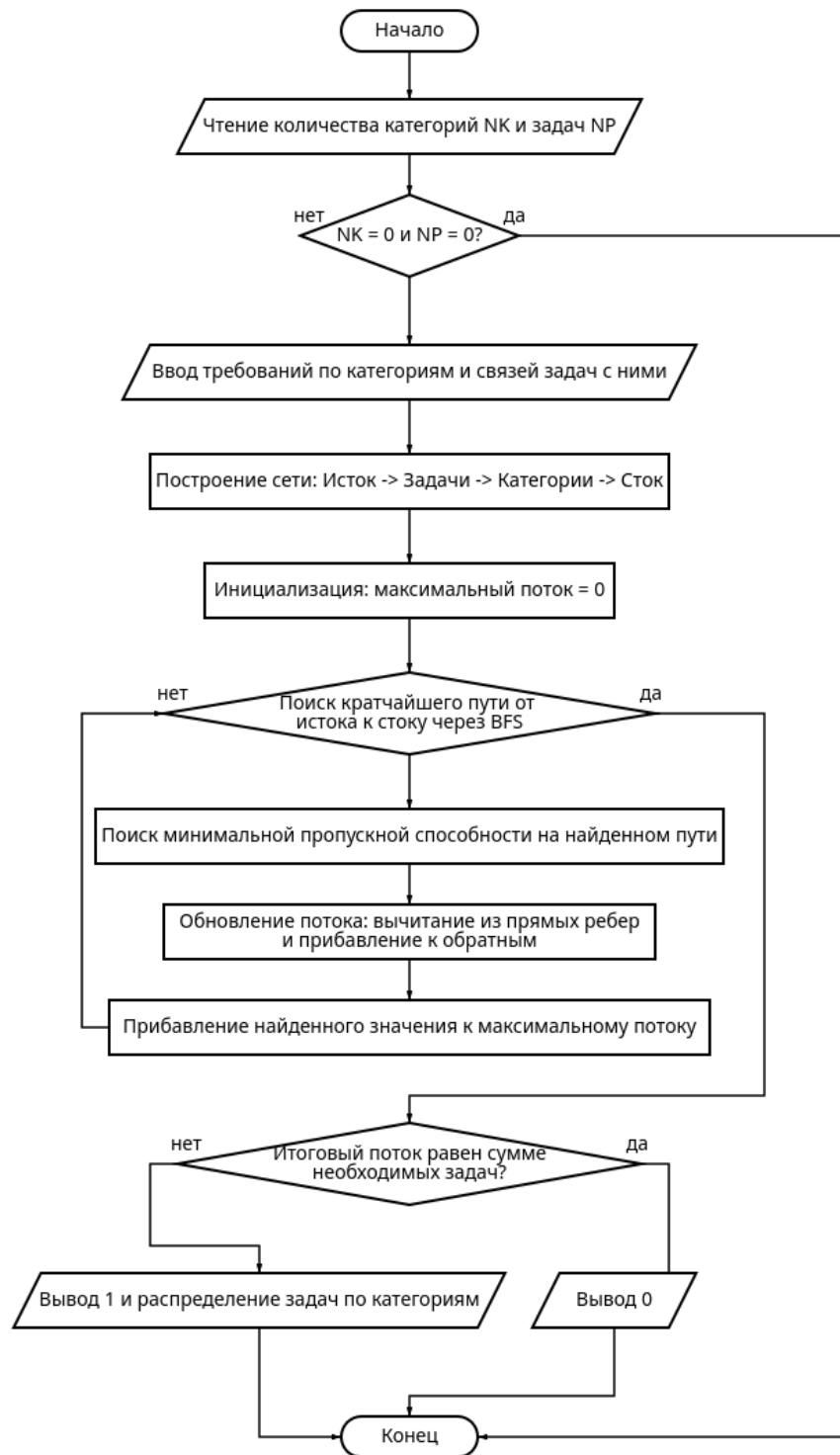


Рисунок 3 - Схема алгоритма.



Тестовые наборы:

Тестовый набор 1

Входные данные:

3 15

3 3 4

2 1 2

1 3

1 3

1 3

1 3

3 1 2 3

2 2 3

2 1 3

1 2

1 2

2 1 2

2 1 3

2 1 2

1 1

3 1 2 3

Выходные данные:

1

14 1 6

9 10 11

2 3 4 5

Тестовый набор 2

Входные данные:

3 14

7 3 4

2 1 2

1 1

1 2

1 2

1 3

3 1 2 3

2 2 3

1 2

1 2

2 2 3

2 2 3

2 1 2

1 1

3 1 2 3

Выходные данные:

0

Тестовый набор 3

Входные данные:

2 3

1 1

1 1

1 2

2 1 2

Выходные данные:

1

1

2

Тестовый набор 4

Входные данные:

2 2

1 1

1 1

1 1

Выходные данные:

0

Тестовый набор 5

Входные данные:

2 2

1 1

2 1 2

1 2

Выходные данные:

1

1

2

Тестовый набор 6

Входные данные:

3 2

1 1 1

3 1 2 3

3 1 2 3

Выходные данные:

0

Тестовый набор 7

Входные данные:

3 5

2 1 2

1 1

2 1 2

2 2 3

2 1 3

1 3

Выходные данные:

1

1 2

3

4 5

Результат прохождения тестов программой:

```
===== test session starts =====
collecting ... collected 7 items

task10_4-tests.py::test_solve_task_setter_status[3-15-categories_needed0-task_to_cats0-1] PASSED [ 14%]
task10_4-tests.py::test_solve_task_setter_status[3-15-categories_needed1-task_to_cats1-0] PASSED [ 28%]
task10_4-tests.py::test_solve_task_setter_status[2-3-categories_needed2-task_to_cats2-1] PASSED [ 42%]
task10_4-tests.py::test_solve_task_setter_status[2-2-categories_needed3-task_to_cats3-0] PASSED [ 57%]
task10_4-tests.py::test_solve_task_setter_status[2-2-categories_needed4-task_to_cats4-1] PASSED [ 71%]
task10_4-tests.py::test_solve_task_setter_status[3-2-categories_needed5-task_to_cats5-0] PASSED [ 85%]
task10_4-tests.py::test_solve_task_setter_status[3-5-categories_needed6-task_to_cats6-1] PASSED [100%]

===== 7 passed in 0.37s =====
```

Рисунок 4 - Скриншот выполнения тестов.

### **Заключение**

В рамках выполненной лабораторной работы были освоены фундаментальные алгоритмы работы с графовыми структурами. Реализованные программные решения позволили закрепить на практике навыки применения методов обхода графов и поиска кратчайших путей.